Práctica. Valorar hallazgos sin confundir indicios y vulnerabilidades

Nombre y apellidos: _____

Jueves 12 de noviembre de 2026 · HE · 4 horas · trabajo individual

La semana pasada sacasteis una lista de hallazgos de los servicios web de TECNORIBERA, cada uno con su estado: confirmado, sospecha, descartado o no evaluado. Hoy hacemos dos cosas con esa lista. Primero, aprender a ponerle una nota a lo que está confirmado, para poder decir qué hay que arreglar antes. Y segundo, estrenar vuestra Metasploitable para ver la diferencia entre lo que una herramienta **dice** que es vulnerable y lo que vosotros **confirmáis** que lo es.

Esta es la pregunta de hoy, y la vais a oír todo el curso: ¿esto es una vulnerabilidad o solo lo parece? Una herramienta que os suelta cuarenta "posibles problemas" no ha hecho el trabajo; lo hacéis vosotros cuando separáis los tres o cuatro que son de verdad y les ponéis número. Un informe que puntúa sospechas como si fueran hechos es peor que no tener informe: hace perder el tiempo a quien tiene que arreglar y quema la confianza del cliente.

1. Qué vas a aprender hoy

Al terminar la sesión tienes que ser capaz de:

1. Explicar la diferencia entre un indicio, una sospecha y una vulnerabilidad confirmada.
2. Confirmar un indicio con una prueba distinta de la que lo generó.
3. Leer y escribir un vector CVSS 3.1 y calcular su puntuación base.
4. Justificar por qué eliges cada métrica del vector.
5. Ordenar los hallazgos por riesgo con una matriz de probabilidad e impacto.
6. Entender por qué un indicio sin confirmar no se puntúa.

2. Antes de empezar

Trae dos cosas hechas:

1. Tu tabla de hallazgos de la práctica del 5 de noviembre (la plantilla de enumeración web). La vamos a usar en la primera parte.
2. La tarea Prepara tu equipo para Metasploitable: Metasploitable importada en tu VirtualBox, tu Kali en la red `labnet` y el fichero `NN-preparacion-metasploitable.txt` con las comprobaciones. Si no la has hecho, avísame al entrar: hoy no dedicamos tiempo de clase a instalar, pero te doy una salida para que no te quedes parado (apartado 11).

Copia a tu equipo el material del laboratorio de hoy desde la web del curso (HE, Prácticas): `cvss31.py` y `plantilla-valoracion.md`. El `cvss31.py` lo usas en el Ubuntu; no necesita Internet ni instalar nada.

Alcance de hoy, como siempre por escrito antes de teclear:

Dentro del alcance: tu máquina Metasploitable en `10.10.10.13`, desde tu Kali en `10.10.10.50`, con las herramientas y las opciones de esta hoja. Lectura y confirmación de versiones y servicios.

Fuera del alcance: cualquier otra IP; abrir una shell y quedarte dentro de la máquina; instalar nada en la víctima; lanzar módulos de explotación que no estén en esta hoja. Hoy confirmamos vulnerabilidades, no las explotamos a fondo: la explotación y la post-explotación son de la UT6, más adelante.

3. Indicio, sospecha, confirmado (20 minutos)

Tres palabras que vais a usar todo el curso. No son lo mismo:

- **Indicio**: algo que te hace pensar que puede haber un problema. "El banner dice versión 2.3.4", "la herramienta ha marcado este puerto en rojo". Un indicio es un punto de partida, no un resultado.
- **Sospecha**: un indicio que encaja con una vulnerabilidad conocida, pero que todavía no has comprobado. "La versión 2.3.4 de ese servicio tuvo una puerta trasera; **podría** tenerla". La palabra clave es *podría*.
- **Confirmado**: lo has comprobado con una prueba, y la prueba ha salido como tenía que salir si la vulnerabilidad existe. Ahora sí es un hallazgo.

La regla de oro: **para confirmar un indicio necesitas una prueba distinta de la que te dio el indicio**. Si nmap te dice la versión por el banner, confirmarlo leyendo otra vez el banner no vale: el banner podría mentir. Hace falta otra prueba que solo salga bien si la vulnerabilidad es real.

Y la consecuencia para hoy: **solo se puntúa lo confirmado**. Un indicio o una sospecha no llevan nota CVSS. Ponerle un 9,8 a algo que no has comprobado es inventar.

4. Ponle nota a lo confirmado: CVSS 3.1 (45 minutos)

CVSS es la forma estándar de ponerle una nota del 0 al 10 a una vulnerabilidad. La usan los boletines de seguridad, los clientes y las herramientas. La nota sale de un **vector**: ocho letras que describen la vulnerabilidad.

4.1. Las ocho métricas

Métrica	Pregunta	Valores
AV — Vector de ataque	¿Desde dónde se ataca?	N red, A red adyacente, L local, P físico
AC — Complejidad	¿Hace falta que se den condiciones especiales?	L baja, H alta
PR — Privilegios	¿Necesita el atacante una cuenta?	N ninguno, L bajos, H altos
UI — Interacción	¿Tiene que hacer algo una víctima (clic, abrir)?	N ninguna, R requerida
S — Alcance (scope)	¿El fallo se queda en el componente o salta a otro?	U no cambia, C cambia
C — Confidencialidad	¿Cuánto expone datos?	H alta, L baja, N ninguna
I — Integridad	¿Cuánto permite modificar?	H alta, L baja, N ninguna
A — Disponibilidad	¿Cuánto afecta al servicio?	H alta, L baja, N ninguna

El vector se escribe pegando las métricas con barras:

```
AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
```

Ese de ejemplo se lee: se ataca por red, es fácil, no hace falta cuenta ni que colabore nadie, el fallo no sale del componente, y compromete del todo la confidencialidad, la integridad y la disponibilidad. Es lo peor: va a dar 9,8.

4.2. La calculadora

En clase no tenemos Internet en la Kali, así que usamos una calculadora que os he preparado. En el Ubuntu:

```
python3 cvss31.py AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
```

Te responde:

```
Vector CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Puntuación base: 9.8  (Crítica)
```

Si la lanzas sin el vector (`python3 cvss31.py` a secas), te pregunta métrica a métrica. Pruébala con estos tres y comprueba que te dan lo mismo que a mí; si no, has copiado mal el vector:

```
python3 cvss31.py AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N     # 6.1, Media
python3 cvss31.py AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H     # 7.8, Alta
python3 cvss31.py AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H     # 7.5, Alta
```

El último es interesante: no expone ni modifica nada (`C:N/I:N`), solo tira el servicio (`A:H`), y aun así es un 7,5. Un fallo que solo deja sin servicio puede ser grave.

4.3. Valora lo tuyo de TECNORIBERA

Abre `plantilla-valoracion.md`, parte A. Coge de tu tabla del 5 de noviembre **solo los hallazgos confirmados** y ponle a cada uno su vector y su nota. Para cada métrica, piensa el caso concreto: por ejemplo, una cabecera de seguridad que falta casi nunca es por sí sola una vulnerabilidad puntuable (lo hablamos la semana pasada); es una ausencia de defensa, y va en el informe, pero sin nota CVSS.

Los que dejaste en sospecha o descartados no se puntúan. Apunta en una frase por qué no llevan nota. Eso también lo evalúo.

5. Estrena tu Metasploitable (15 minutos)

Arranca el laboratorio desde el Ubuntu (no desde la Kali):

```
cd ~/Descargas          # o donde tengas labnet-up.sh
./labnet-up.sh comprobar
./labnet-up.sh arrancar
```

`comprobar` revisa que Metasploitable solo ve la red `labnet` y que tu Kali está dentro; si algo falla, te lo dice y no sigas. `arrancar` la enciende sin ventana. Espera un minuto. Luego, **desde tu Kali**:

```
ping -c 2 10.10.10.13
```

Tienen que volver las dos respuestas. Si no vuelven, mira el apartado 11. No cambies la configuración de red de las VM por tu cuenta.

6. De los servicios a los indicios (40 minutos)

Mira qué hay escuchando y con qué versión:

```
nmap -sV -Pn -p- --open -oN salidas/nmap-metasploitable.txt 10.10.10.13
```

`-sV` intenta averiguar el servicio y la versión de cada puerto, `-p-` mira los 65535 puertos, `--open` solo enseña los abiertos, `-Pn` se salta el ping. Tarda un poco. Vas a ver muchos puertos: FTP, SSH, telnet, servidores web, bases de datos, Samba y más. Es una máquina hecha para ser vulnerable.

Cada línea de esa salida es un **indicio**, no un hallazgo. Fíjate en la columna `VERSION`. Apunta en la plantilla (parte B) tres servicios que te llamen la atención por tener una versión concreta y antigua. Candidatos buenos, por lo claros que son de confirmar: el FTP del puerto 21, el Samba de los puertos 139 y 445, y el servidor web.

Ahora, para tener la lista de "posibles problemas" conocidos de esas versiones, usamos una base de datos de exploits que la Kali trae **en local**, sin Internet:

```
searchsploit vsftpd 2.3.4
searchsploit samba 3.0
```

`searchsploit` busca en la copia local de Exploit-DB. Lo que te devuelve son **sospechas**: "para esta versión existe esto". Que exista un exploit publicado no quiere decir que *esta* máquina sea

vulnerable: puede estar parcheada, configurada de otra forma o ser una versión que solo se le parece. Por eso lo siguiente es confirmar.

7. Confirma dos o tres, sin pasarte (50 minutos)

Vas a confirmar los indicios con pruebas de **lectura o comprobación**, no abriendo sesiones ni quedándote dentro de la máquina. Elige al menos dos de estos tres. Apunta la hora y guarda la salida de cada comprobación.

7.1. El FTP del puerto 21

El indicio es la versión que anuncia. Confírmalo de dos maneras independientes.

Primero, ¿deja entrar sin contraseña (FTP anónimo)? Eso es un problema real y se comprueba sin explotar nada:

```
nmap -p 21 --script ftp-anon -oN salidas/ftp-anon.txt 10.10.10.13
```

Si el script responde que el acceso anónimo está permitido, lo has **confirmado**: es un hecho, no una sospecha.

Segundo, el indicio de la versión con puerta trasera. Hay un script de nmap que **comprueba** si la puerta trasera responde, sin dejarte una shell abierta:

```
nmap -p 21 --script ftp-vsftpd-backdoor -oN salidas/ftp-backdoor.txt
10.10.10.13
```

Lee con cuidado lo que dice el script: si marca el estado como vulnerable, es que ha comprobado la condición, y eso confirma el indicio. Si no, el indicio se queda en sospecha. En ningún caso te metas dentro de la máquina: hoy confirmamos, no explotamos.

7.2. El Samba de los puertos 139 y 445

El indicio es la versión. Confirma la versión con una prueba distinta del banner:

```
nmap -p 139,445 --script smb-protocols,smb-os-discovery \
  -oN salidas/smb.txt 10.10.10.13
```

Estos scripts preguntan al propio servicio por los protocolos que habla y por el sistema. Si la versión que sacas aquí coincide con la de nmap `-sV`, tienes dos fuentes que dicen lo mismo: versión confirmada. Si `searchsploit samba 3.0` te daba sospechas para esa versión, ahora ya puedes decir que la versión es la que es; lo que no has comprobado es que el fallo concreto sea explotable, así que eso se queda como sospecha bien fundada, y así lo escribes.

7.3. El servidor web

```
curl -sI http://10.10.10.13/ | tee salidas/web-cabeceras.txt
whatweb -a 1 --color=never http://10.10.10.13/ | tee salidas/web-whatweb.txt
```

Compara la versión del `Server` de las cabeceras con la que dice WhatWeb y con la de nmap. Tres fuentes. Si coinciden, versión confirmada. Apunta qué páginas o aplicaciones de ejemplo ves, porque esa máquina trae varias webs vulnerables montadas y las usaremos en la UT8.

7.4. Pon nota a lo confirmado

Para cada cosa que hayas **confirmado**, escribe el vector CVSS y calcula la nota con `cvss31.py`. Piensa cada métrica para el caso. Por ejemplo, el FTP anónimo: se ataca por red (`AV:N`), es fácil (`AC:L`), no hace falta cuenta (`PR:N`) ni que colabore nadie (`UI:N`); lo que expone depende de qué haya en ese FTP. Razona el resto tú y apúntalo en la columna "por qué este vector".

Lo que se haya quedado en sospecha va a la plantilla, en su columna, **sin nota**.

8. Ordena por riesgo (25 minutos)

La nota CVSS mide la vulnerabilidad en sí, pero para ordenar tu informe también cuenta cómo de probable es que alguien la aproveche aquí. Coloca cada hallazgo confirmado en la matriz de la plantilla (probabilidad × impacto) y saca la lista final, de lo más urgente a lo menos. No siempre el de nota más alta va primero: un fallo de nota media muy fácil de aprovechar y expuesto puede ir antes que uno crítico que requiere condiciones raras. Escribe en una frase por qué has puesto primero el que has puesto.

9. Cierra el laboratorio (5 minutos)

Deja la máquina como estaba, para la próxima y para no gastar recursos:

```
./labnet-up.sh restaurar
./labnet-up.sh parar
```

`restaurar` devuelve Metasploitable a la instantánea limpia (deshace cualquier cambio que hayas provocado) y `parar` la apaga. Comprueba con `./labnet-up.sh estado` que queda apagada.

10. Preguntas

1. Explica con tus palabras la diferencia entre un indicio y una vulnerabilidad confirmada, con un ejemplo de hoy.
2. ¿Por qué no vale confirmar la versión de un servicio leyendo otra vez el mismo banner?
3. De los hallazgos que has confirmado hoy, ¿cuál tiene la nota CVSS más alta y por qué le has puesto esas métricas?
4. ¿Por qué un indicio sin confirmar no lleva nota CVSS?
5. Pon un ejemplo de un hallazgo con nota media que ordenarías por delante de uno con nota alta, y di por qué.

11. Si algo no funciona

`./labnet-up.sh comprobar` dice que algo está mal: haz lo que te diga el mensaje (normalmente `./labnet-up.sh cablear`). Si no se arregla, avísame; no toques la red de las VM a mano.

`ping 10.10.10.13` no responde: comprueba que Metasploitable está encendida (`./labnet-up.sh estado`) y que tu Kali tiene una IP `10.10.10.x` (`ip -br a` en la Kali). Si la Kali no tiene esa IP, su tarjeta de `labnet` no está conectada; dímelo.

No has podido traer la Metasploitable: no te quedes parado. Haz entera la parte A (valoración de TECNORIBERA con CVSS), que no necesita la víctima, y la parte teórica de la B: con la salida de `nmap -sV` que te paso yo impresa, escribe qué es indicio y qué confirmarías y cómo. Luego, en casa o en una clase de recuperación, montas la máquina y completas las comprobaciones.

`searchsploit` dice que no encuentra la base de datos: en la Kali, `searchsploit -u` la actualiza, pero necesita Internet; para hoy, con la que trae de serie basta. Si ni así responde, apúntalo y sigue con las comprobaciones de nmap, que no dependen de searchsploit.

`nmap -p-` tarda muchísimo: es normal con los 65535 puertos. Si tienes prisa, para esta clase puedes limitarlo a los puertos típicos quitando `-p-` (nmap mira los 1000 más comunes) y lo apuntas como limitación.

12. Qué tienes que entregar (20 minutos)

Carpeta `NN-HE-valoracion` en ZIP con:

1. `plantilla-valoracion.md` completa: parte A (TECNORIBERA con CVSS), parte B (Metasploitable: indicios, confirmaciones, CVSS) y la matriz de riesgo con la lista final ordenada.
2. La carpeta `salidas/` con los ficheros de las comprobaciones que hayas hecho (nmap, scripts, curl, whatweb).
3. Las respuestas a las preguntas del apartado 10.

Si no has podido traer la máquina, entrega la parte A completa y la parte B en versión teórica, con la explicación de qué comprobarías. Cuenta.

13. Antes de entregar, comprueba que

1. Cada hallazgo con nota CVSS está confirmado con una prueba tuya, no con lo que dice una herramienta.
2. Los indicios sin confirmar están como sospecha, sin nota.
3. Cada vector CVSS tiene al lado la razón de sus métricas.
4. La lista final está ordenada por riesgo y has explicado el primero.
5. Has restaurado y apagado la Metasploitable (`./labnet-up.sh estado` la muestra apagada).

14. Seguridad y ética

Todo lo de hoy es contra tu propia máquina, en tu propio ordenador, en una red interna que no sale de ahí. Las mismas órdenes contra un equipo ajeno sin permiso por escrito son un delito, también las de "solo mirar". Fíjate en que hoy hemos confirmado vulnerabilidades sin llegar a explotarlas del todo: en una auditoría real, hasta dónde puedes llegar lo marca el alcance firmado, y confirmar que algo es explotable sin reventar el sistema del cliente es parte del oficio.

La semana que viene empezamos el proyecto de verdad: cada uno pondrá en marcha **su propia web** (la misma que usaréis en Puesta en Producción Segura y vigilaréis con Wazuh en Incidentes) y la auditaréis. Lo veremos el martes en clase; hoy no hay que preparar nada para eso.

Referencias

- Especificación CVSS 3.1 (FIRST): https://www.first.org/cvss/v3.1/specification-document
- Calculadora oficial CVSS 3.1: https://www.first.org/cvss/calculator/3.1
- Metasploitable 2, guía de Rapid7: https://docs.rapid7.com/metasploit/metasploitable-2/
- Scripts NSE usados hoy: https://nmap.org/nsedoc/scripts/ftp-anon.html · https://nmap.org/nsedoc/scripts/ftp-vsftpd-backdoor.html ·

https://nmap.org/nsedoc/scripts/smb-os-discovery.html

- Exploit-DB y SearchSploit: https://www.exploit-db.com/searchsploit